

CHAP 11(1): numpy 배열

numpy 배열이란, numpy 설치, numpy 사용하기
numpy 배열 생성,
numpy 배열의 연산(산술 연산, 관계 · 논리 연산)
numpy 1차원 배열의 인덱싱, 슬라이싱
numpy 2차원 배열의 인덱싱, 슬라이싱

[참고도서]

- 따라하며 배우는 파이썬과 데이터과학
- 데이터 분석을 위한 파이썬 철저 입문

numpy 배열이란?

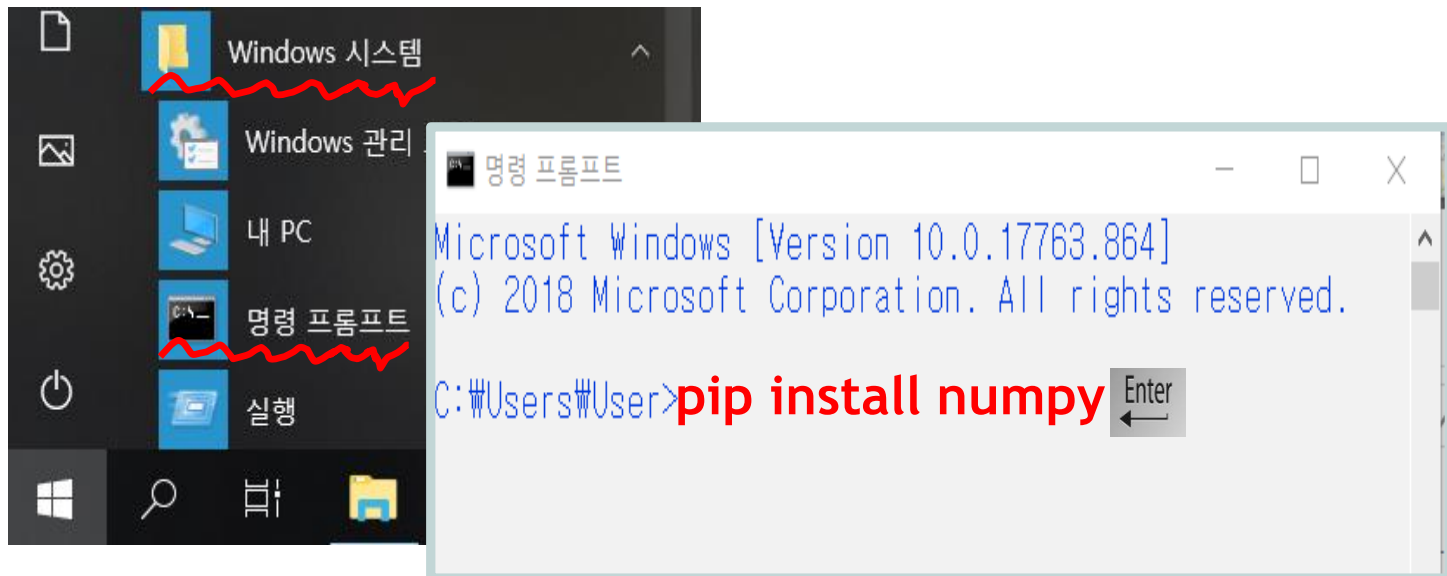
- 많은 데이터를 리스트 변수에 넣고 관리하면, 속도가 느리고 메모리를 많이 차지하는 단점이 있다.
- 배열(array)을 사용하면,
 - 메모리를 적게 차지하고, 처리 속도가 빠르다.
 - 배열은 같은 종류의 데이터를 연속으로 저장한다.
따라서, 원소에 대한 접근과 반복 실행이 빨라진다.
- 배열 라이브러리 패키지 numpy를 설치하고, import한다.
(파이썬은 배열 자료형을 제공하지 않음)
- numpy(넘파이, 넘피) 배열 자료형
 - 다차원의 배열 자료구조 클래스인 ndarray를 지원.
 - ndarray 객체- numpy를 통해 생성되는 n차원의 배열 객체, 동일한 자료형의 데이터만을 담을 수 있다.
 - 벡터화 연산(vectorized operation)을 지원.
 - 하나의 명령어를 배열의 모든 원소에 적용하여 병렬적으로 연산
 - 배열과 배열 간에 다양한 수학적 연산이 가능.
 - 배열 인덱싱(array indexing)을 사용한 질의(Query) 기능으로 복잡한 수식을 계산 가능함.

numpy 설치

- numpy
 - 다차원 배열을 처리하는데 필요한 여러 유용한 기능을 제공.
 - numpy 배열(ndarray 객체)은 같은 종류의 데이터만을 담을 수 있다.
- numpy 라이브러리를 install 한 후 사용한다.
 - 윈도우 콘솔창(커맨드창)에서 install을 실행

```
pip install numpy # install 명령어
```

- 윈도우 콘솔창은 <Windows 시스템> → <명령 프롬프트>



numpy 사용하기

- 배열을 사용하기 위해서 `numpy` 라이브러리를 `import`한다.

```
import numpy as np # import 문. numpy를 별칭 np 로 사용
```

- 1차원 배열
 - 수치 정보 중심으로 구성, 데이터가 하나의 줄로 나열.
 - 레이블이 없다.
- 2차원 배열을 **행렬(matrix)**이라고 함.
 - 1차원 배열을 여러 줄로 겹쳐 놓은 형태.
 - 2가지 방향으로 각각 위치 값이 필요.
가로줄은 **행(row)**, 세로줄은 **열(column)**이라고 함.
 - 행과 열에 레이블이 없다.(한계)
- `numpy` 배열 객체는 속성을 갖고 있음. 따라서 배열 정보 확인이 가능.
 - `ndim` 배열의 차원
 - `shape` 각 차원의 크기(튜플로 표시)
 - `size` 전체 요소의 개수
 - `dtype` 배열에 저장된 자료형

[참고자료] numpy 배열(ndarray 객체)

배열 객체 안에는
배열데이터와 메소드(함수),
속성(변수) 등이 포함되어 있다.

`a = np.array( [1,2,4] )`

`배열.속성`

`배열.함수( )`

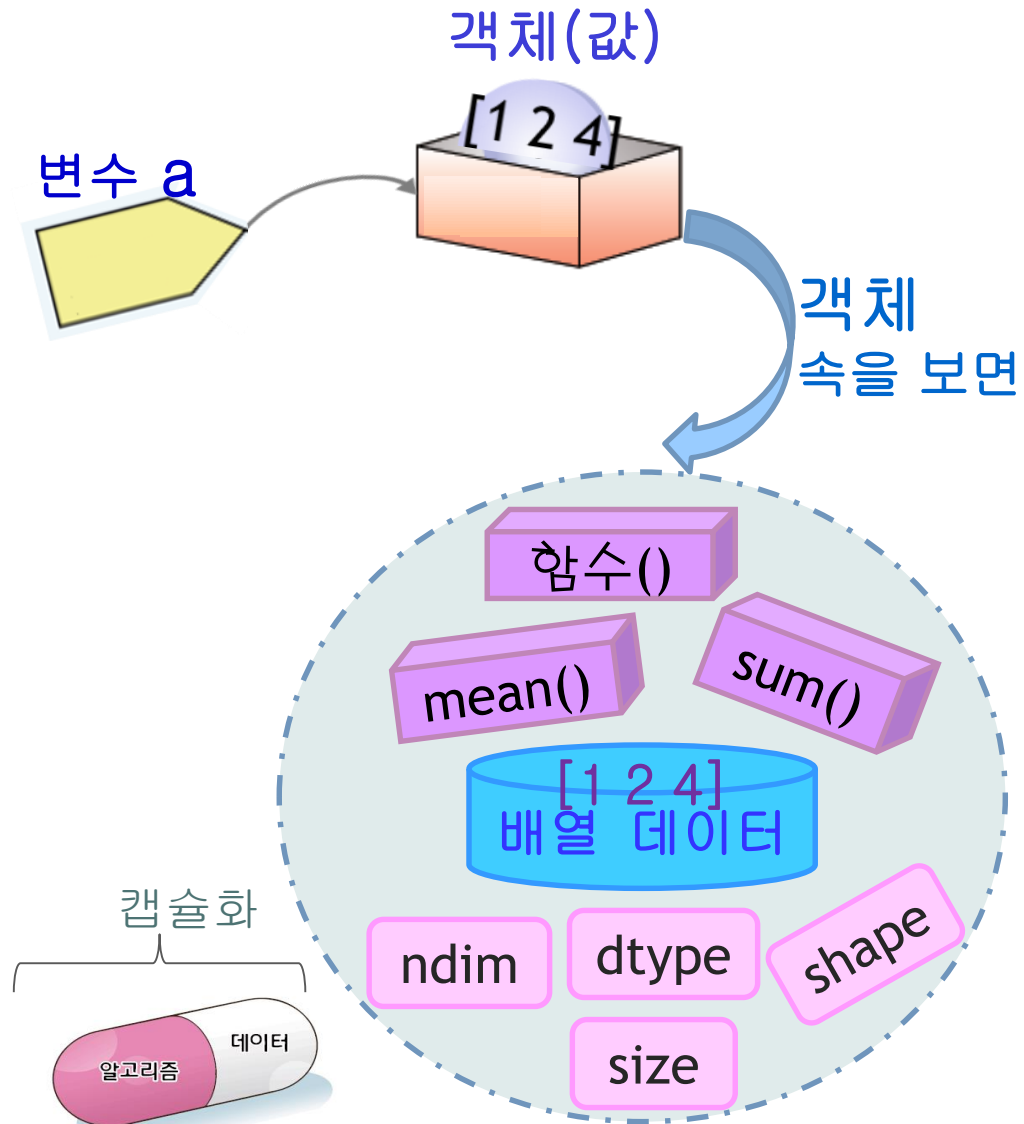
`a.ndim`

`a.size`

`a.shape`

`a.dtype`

`a.mean( )`

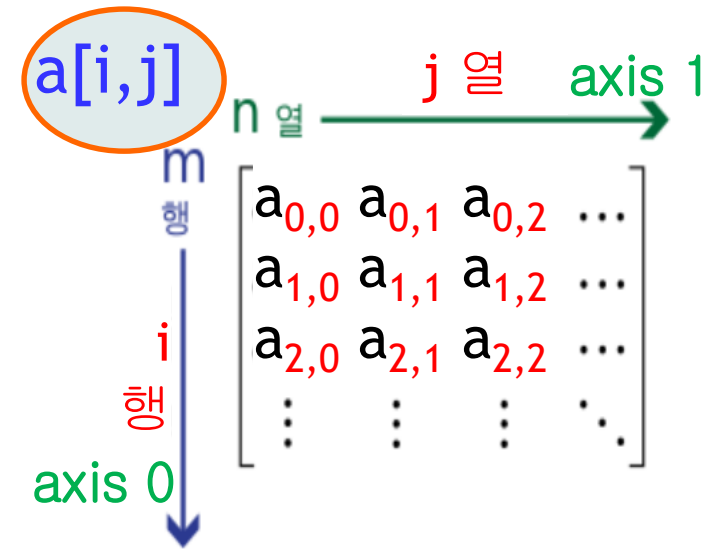


출처: Power JAVA

[참고자료] 2차원 배열

- 파이썬에서, 2차원 배열

```
a= numpy.array(  
    [[a00, a01, ... , a0n-1 ], #a[0]. 0번 행  
     [a10, a11, ... , a1n-1 ], #a[1]. 1번 행  
     .....  
     [am-10, am-11, ... , am-1n-1 ]  
     #a[m-1]. m-1번 행  
 )
```



비교. 리스트

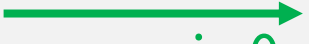
a가 리스트이면 $a[i][j]$

```
a= [ [a00, a01, ... , a0n-1 ],  
      [a10, a11, ... , a1n-1 ],  
      .....  
      [am-10, am-11, ... , am-1n-1 ] ]
```

numpy 배열 생성

- numpy 배열 생성하기.
 - ① array()함수의 인수에 리스트를 지정하여 배열 생성.
 - ② arange(start,end,step)함수로 배열 생성.
- ① array()함수의 인수에 리스트를 지정
 - numpy의 array()함수에 인수로 리스트를 지정하여 numpy 배열 생성
 - 예1) 1차원 배열 생성

```
import numpy as np
data = [1, 3, 5, 7, 9]
a = np.array( data ) # 1차원 배열 생성. ndarray객체
# a= np.array( range(1,10, 2) )와 동일
print(a.ndim) # 1
print(a.size) # 5
print(a.shape) # (5, )
print(a.dtype) # dtype('int32')
print(a.mean()) # 5.0
print( a ) # [1 3 5 7 9]
```

```
>>> a 
array( [1, 3, 5, 7, 9] )
>>> 
axis 0
```

😊출력 화면

```
1
5
(5, )
dtype('int32')
5.0
[ 1 3 5 7 9]
```

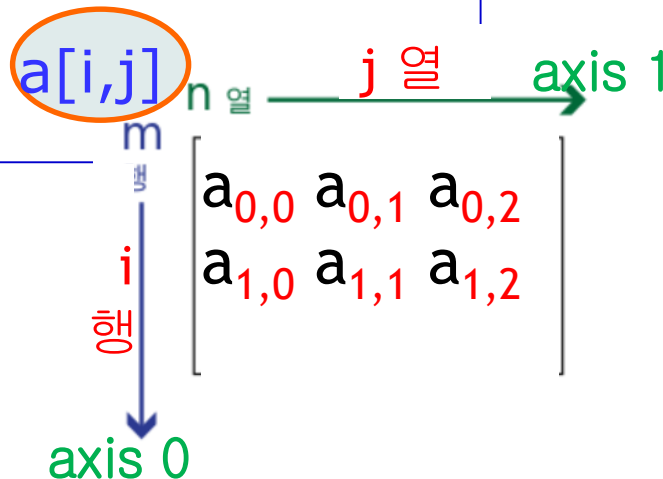
numpy 배열 생성

- 예2) 2차원 배열 생성하기. 배열 b는 2x3 배열.

```
import numpy as np

# 리스트를 사용하여 배열 생성
a = np.array( [ [1,2,3], [4,5,6] ] )

print( a[0] )      # [1 2 3]
print( a[1] )      # [4 5 6]
print( a[0,0] )    # 1
print( a[1,2] )    # 6
print( a )
```



😊 출력 화면

```
[ 1 2 3] # a[0]
[ 4 5 6] # a[1]
1       # a[0,0]
6       # a[1,2]
[[ 1 2 3] # a
 [ 4 5 6]]
```

```
>>> a
array([ [1, 2, 3],
        [4, 5, 6]])
>>>
```

numpy 배열 생성

- ② arange()함수를 사용하여 numpy 배열 만들기.
 - **arange(start,end,step)** - range()와 비슷함. 배열 객체 생성.
 - 예1) 1부터 20까지의 정수 값을 가지는 ndarray객체 생성하기

```
import numpy as np  
  
a= np.arange(1,21)  
    #np.array( range(1,21) ) 동일  
  
print(a)
```

😊 출력 화면

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15  
 16 17 18 19 20]
```

numpy 배열 생성

- ③ numpy가 제공하는 여러 함수들로 배열 객체 생성 가능.
 - `zeros((행, 열))` - 해당 배열에 모두 0을 집어 넣어 생성
 - `ones((행, 열))` - 모두 1을 집어 넣는다.
 - `full((행, 열), 값)` - 배열에 사용자가 지정한 값을 넣는다
 - `eye(행)` - 대각선은 1, 나머지는 0을 갖는 행과 열이 같은 배열 생성.
 - `linspace(x1, x2, n)` - 구간 `[x1, x2]` 안에서 균일한 간격의 데이터를 `n`개 만들어 배열을 생성. 디폴트는 50개.
`x1` 시작값, `x2` 끝값, `n` 데이터 생성 개수
 - `reshape(행, 열)` - 배열을 지정한 형태로 변형하는 함수.

numpy 배열 생성

- 예1) 구간 내의 균일한 간격의 값들을 사용하여 배열 생성.

```
>>> import numpy as np
>>> y = np.linspace(5, 10) # 구간 [5,10] 내의 균일한 간격의 값 50개
>>> y
array([ 5.          ,  5.10204082,  5.20408163,  5.30612245,  5.40816327,
        5.51020408,  5.6122449 ,  5.71428571,  5.81632653,  5.91836735,
        6.02040816,  6.12244898,  6.2244898 ,  6.32653061,  6.42857143,
        6.53061224,  6.63265306,  6.73469388,  6.83673469,  6.93877551,
        7.04081633,  7.14285714,  7.24489796,  7.34693878,  7.44897959,
        7.55102041,  7.65306122,  7.75510204,  7.85714286,  7.95918367,
        8.06122449,  8.16326531,  8.26530612,  8.36734694,  8.46938776,
        8.57142857,  8.67346939,  8.7755102 ,  8.87755102,  8.97959184,
        9.08163265,  9.18367347,  9.28571429,  9.3877551 ,  9.48979592,
        9.59183673,  9.69387755,  9.79591837,  9.89795918, 10.          ])
>>> y2= np.linspace(2, 3, 5) # 구간 [2,3] 내의 균일한 값 5개를 사용
>>> y2
array([2.   , 2.25, 2.5 , 2.75, 3.   ])
>>>
```

numpy 배열 생성

😊출력 화면

- 예2)

```
import numpy as np

a = np.zeros((2,2))
print(a) ; print()

b = np.ones((2,3))
print(b)

c = np.full((2,3), 5)
print(c) ; print()

d = np.eye(3) #행과 열이 3
print(d) ; print()

e = np.arange(1,21)
f = e.reshape((4,5)) #4행5열
print(f)
```

```
[ [0. 0.]      # a
  [0. 0.] ]

[ [1. 1. 1.]   # b
  [1. 1. 1.] ]

[ [5 5 5]      # c
  [5 5 5] ]

[ [1. 0. 0.]   # d
  [0. 1. 0.]
  [0. 0. 1.] ]

[ [ 1 2 3 4 5]   # f
  [ 5 6 7 8 10]
  [11 12 13 14 15]
  [16 17 18 19 20] ]
```

numpy 배열의 연산

- numpy 배열 연산
 - 배열의 각 원소에 대한 반복 연산을 하나의 명령어로 처리하는 **벡터화 연산(vectorized operation)**을 지원.
 - 따라서, 계산 속도도 반복문을 사용할 때보다 훨씬 빠름
 - 산술 연산, 관계 연산과 논리 연산 등 모든 연산에 적용된다
 - 배열간 연산을 실행할 수도 있다.
(예) 배열 **a**와 **b**가 있을 때, **a + b**는 각 배열요소의 합을 계산.
a[0]+b[0], a[1]+b[1],...과 같은 방식으로 계산하고 결과를 반환.
 - +, -, *, / 등의 연산자를 사용할 수 있다.
 - 배열의 각 원소에 대한 반복 연산을 하나의 명령어로 처리.
 - **add(), subtract(), multiply(), divide()** 등의 함수 사용
 - **mean()** #평균 계산
 - **median()** #중앙값 계산

numpy 배열의 연산

- 예) 배열에서 + 연산
 - 배열의 각 원소에 대해 반복 연산을 처리.

```
>>> a = np.array( [1,2,3] )  
>>> b = np.array( [4,5,6] )
```

각 원소 더하기

```
>>> c = a + b # 대응되는 원소 끼리 병렬 연산  
>>> print(c)  
[5 7 9]  
>>>
```

$c[0]=a[0]+b[0]$
 $c[1]=a[1]+b[1]$
 $c[2]=a[2]+b[2]$

(비교) 리스트에서 +연산

```
>>> a = [1,2,3]  
>>> b = [4,5,6]  
>>> a + b #연결 연산  
[1, 2, 3, 4, 5, 6]  
>>>
```

```
# 원소 끼리 덧셈 연산하기  
c=[]  
for i in range(len(a)):  
    hap= a[i]+b[i]  
    c.append(hap)
```

numpy 배열의 연산

- 예) 원소를 모두 2배 하기

① 1차원 배열로 만들어 연산하기

```
a = np.array( [1,2,3] ) # 배열 생성
```

```
a = 2 * a
```

```
print(a) # [2 4 6]
```

배열의 각 원소에 대한 반복 연산을 하나의 명령어로 처리.
따라서, 계산 속도가 훨씬 빠르다.

비교

② 리스트에서는 for문 사용

```
a = [ 1, 2, 3]
```

```
for i in range(len(a)) :
```

```
    a[i]= a[i]*2
```

```
print(a) # [2, 4, 6]
```

numpy 배열의 연산

- 예) 배열에서 산술 연산하기
 - 하나의 명령어로 배열의 각 원소에 대한 반복 연산을 처리.

```
a = np.array( [1,2,3] )  
b = np.array( [4,5,6] )
```

```
# 각 원소 끼리 빼기
```

```
c = a - b  
c = np.subtract(a, b)  
print(c)    # [-3 -3 -3]
```

```
# 각 원소 끼리 곱하기
```

```
c = a * b  
c = np.multiply(a, b)  
print(c)    # [4 10 18]
```

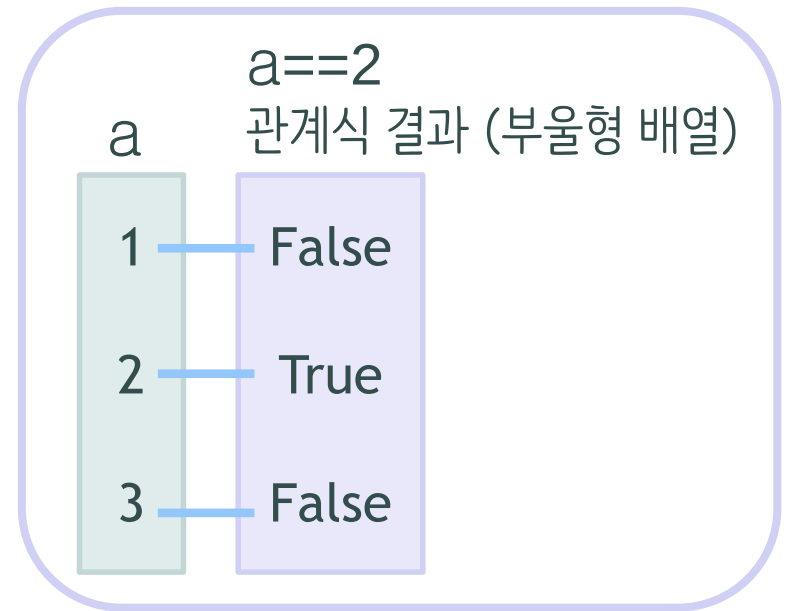
```
# 각 요소 끼리 나누기
```

```
c = a / b  
c = np.divide(a, b)  
print(c)    # [0.25 0.4 0.5]
```

numpy 배열의 연산

- 1차원 배열에서 관계 연산, 논리 연산
 - 하나의 명령어로 배열의 각 원소에 대해 반복 연산을 처리.
 - 연산 결과는 1차원 부울형(Boolean) 배열.
 - 예)

```
>>> a = np.array( [1,2,3] )
>>> b = np.array( [10, 20, 30] )
>>> a == 2
array( [False, True, False] )
>>>
>>>
>>> b > 10
array( [False, True, True] )
>>> (a==2) & (b >10)
array( [False, True, False] )
>>>
```



numpy 배열의 연산

- 2차원 배열에서 관계 연산, 논리 연산
 - 하나의 명령어로 배열의 각 원소에 대해 반복 연산을 처리.
 - 연산 결과는 부울형(**Boolean**) 배열.
 - 예) 배열 전체에서 짝수 원소 찾기, 2열 전체에서 5보다 큰 원소 찾기

```
>>> a= np.array( [[ 1, 2, 3],  
                  [ 4, 5, 6],  
                  [ 7, 8, 9] ] )
```

```
>>> a%2 == 0
```

```
array([[False,  True, False],  
       [ True, False,  True],  
       [False,  True, False] ] )
```

```
>>>
```

```
>>> a[:, 2] > 5
```

```
array([False,  True,  True])
```

```
>>>
```

배열 a의 원소를 2로 나눠
나머지가 0인 경우에는 True,
아닌 경우에는 False 값이 놓인
배열이 생성됨.

모든 행, 2열의 원소가
5보다 큰 경우에는 True,
아닌 경우에는 False 값이
놓인 배열이 생성됨.

numpy 배열의 연산

- $a \% 2 == 0$
 - 주어진 관계 연산이 참인 배열의 원소 자리에는 **True**, 거짓인 원소 자리엔 **False** 값이 놓인다.
 - 이렇게 구성한 새 배열, 부울형 배열이 연산 결과.

a	$a \% 2 == 0$ 관계식 결과 (부울형 배열)
[1, 2, 3]	[False, True, False],
[4, 5, 6]	[True, False, True],
[7, 8, 9]	[False, True, False]

관계연산식이 참인 항목에는 True,
거짓인 항목에는 False 값이 놓인
부울형 배열이 생성.

numpy 배열의 연산

- `sum(배열, axis= n)`
 - 각 배열 요소들을 더하는데 사용하는 함수
 - 선택옵션으로 **axis** 을 지정할 수 있다.
 - **axis**가 0 이면 열끼리 더하는 것.
 - **axis**가 1 이면 행끼리 더하는 것.
- `prod(배열, axis= n)`
 - 각 배열 요소들을 곱하는데 사용
 - 선택옵션으로 **axis** 을 지정할 수 있다.
 - **axis**가 0 이면 열끼리 곱한다.
 - **axis**가 1 이면 행끼리 곱하고

numpy 배열의 연산

- 예)

```
import numpy as np

a = np.array([[1,2],[3,4]])

s = np.sum(a) #모든 원소를 더함
print(s) # 10

# axis=0 이면, 열끼리 더함
# axis=1 이면, 행끼리 더함
s = np.sum(a, axis=0)
print(s) # [4 6]

s = np.sum(a, axis=1)
print(s) # [3 7]

s = np.prod(a)
print(s) # 24
```

생성된
2차원 배열 객체

```
>>> a
array([ [1, 2],
        [3, 4] )
>>>
```

😊 출력 화면

```
10
[4 6]
[3 7]
24
```

numpy 1차원 배열의 인덱싱

- 1차원 배열에서, 원소 참조
 - 리스트와 비슷하게 인덱스를 사용.

변수[i] # i는 인덱스. 0,1,2 지정된 원소 값 참조.

- 예) 1차원 배열 a의 원소 참조하기.

```
>>> import numpy as np
>>> data = [1, 3, 5, 7, 9]
>>> a = np.array( data ) #1차원 배열 생성
>>> a[1] #원소 값 참조      a = np.array( [1, 3, 5, 7, 9] )
3
>>> a[1] = 10 #원소 값 수정
>>> a[1]
10
>>>
```

numpy 1차원 배열의 슬라이싱

- 1차원 배열에서, 부분 값 추출 (슬라이싱)
 - 리스트와 비슷함. 인덱스를 사용.

변수 `[[i1, i2, ..., in]]` `# i는 인덱스. 지정된 원소들을 선택 추출.`
`# 추출 값은 배열의 부분 값 .`

변수 `[start : end]`

- 예)

```
>>> import numpy as np
>>> data = [1, 3, 5, 7, 9]
>>> a = np.array( data ) #1차원 배열 생성
>>> a[[1]] #원소 a[1] 선택, 추출. 슬라이싱
array([3])
>>> a[ [1,2,3] ] # 원소 a[1],a[2],a[3] 선택, 부분 추출
array([3, 5, 7])
>>> a[1:4] # a[1],a[2],a[3] 을 슬라이싱. 부분 추출
array([3, 5, 7])
>>>
```

} 동일

numpy 1차원 배열의 슬라이싱

- 1차원 배열에서 부분 집합 구하기: **부울형 인덱싱 (boolean indexing)**
 - **부울형 배열을 인덱스로** 사용하여 특정 조건의 데이터만 필터링
 - 부울형 배열을 만들고, 인덱스로 지정한다.
배열 각 요소의 선택 여부를 **True, False**로 표현하는 방식.
 - **True**인 요소만 추출 함.
 - 부울형 배열은
 - **True, False**로 이루어진 배열.
 - 추출하는 배열과 크기가 같아야 함.
 - 원소의 추출 여부를 **True, False**로 표현.
 - 선택할 원소는 **True**, 그렇지 않으면 **False**로 표현.
 - 부울형 배열은 배열의 관계연산으로 보통 구성한다.
배열의 관계연산은 **True, False**로 이루어진 배열을 반환,
이것을 인덱스로 사용하여 특정 값을 추출한다.
- 인덱스로 지정된 부울형 배열의 **True**와 대응되는 요소만 추출.

numpy 1차원 배열의 슬라이싱

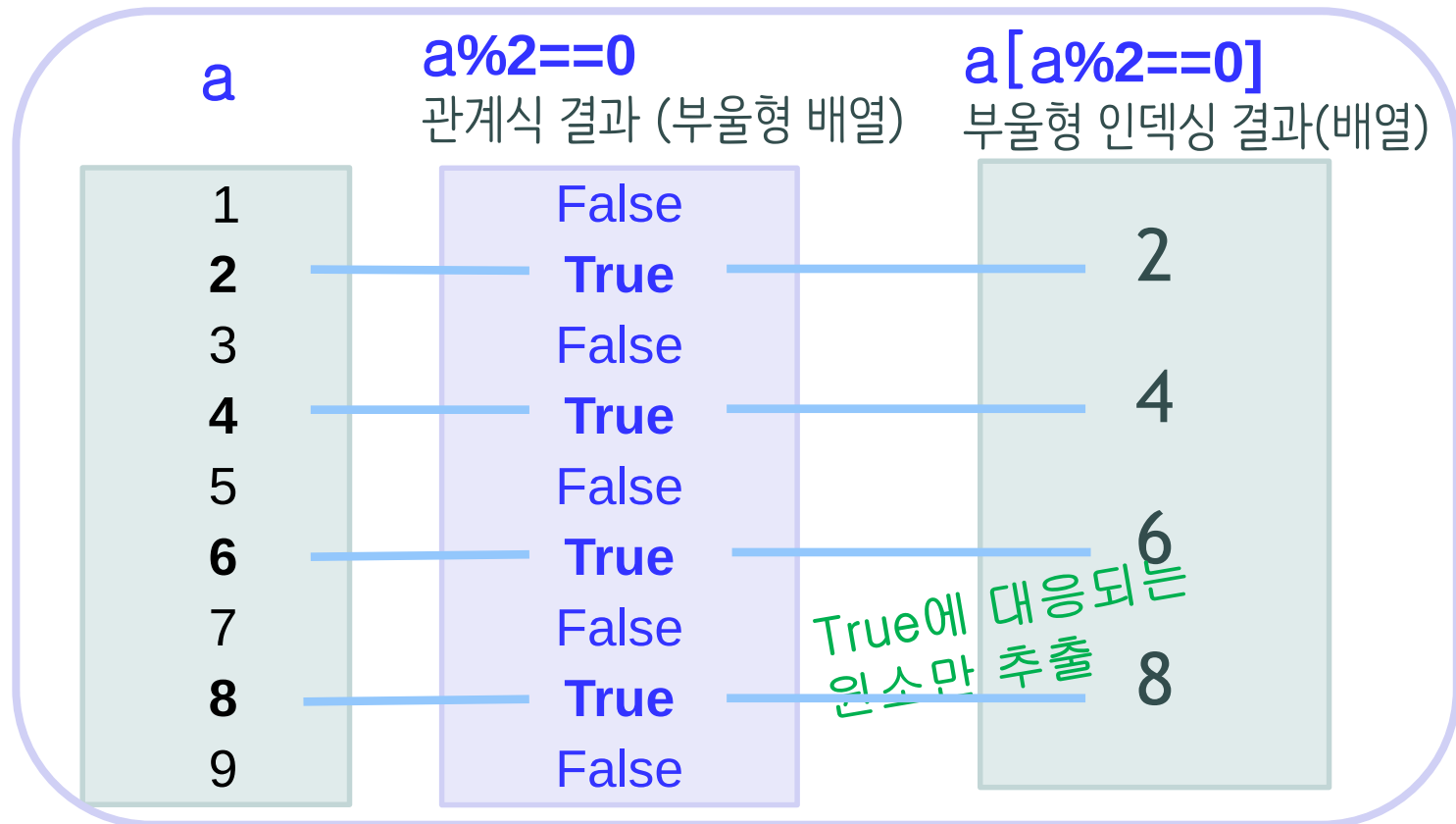
- 예) numpy 1차원 배열에서 짝수인 원소만 추출하기.

```
>>> a= np.array( [ 1, 2, 3, 4, 5, 6, 7, 8, 9 ] )
>>> a[ a%2 == 0 ] # boolean indexing
array([2, 4, 6, 8]) # boolean 배열을 인덱스로 사용하여,
>>>                               # True인 원소에 대응되는 값만 추출.
>>> a % 2 == 0 
array([ False,  True,  False,  True,  False,  True,
        False,  True,  False])
>>>
```

부울형 배열 생성.
배열 a의 원소가
짝수이면 True,
홀수이면 False인 배열.

numpy 1차원 배열의 슬라이싱

- `a[a%2==0]`
 - 부울형 인덱싱 (boolean indexing)으로 조건에 맞는 배열의 원소들을 추출한다.
 - 결과는 `array([2, 4, 6, 8])`



numpy 2차원 배열의 인덱싱

- 2차원 배열에서 특정 원소를 꺼내는 방법
 - 특정 행 전체를 참조할 경우, 행 번호(행 인덱스)를 지정.
변수[행 번호]
 - 특정 원소 1개를 참조할 경우, 인덱스 2개 필요.
첫 번째 인덱스는 행 번호, 두 번째 인덱스는 열 번호.
변수[행 번호, 열 번호]
- 예) **$a[i]$** #행 참조. i 는 행 번호. 0, 1, 2
 $a[i,j]$ #특정 원소 참조. i 는 행 번호, j 는 열 번호

```
>>> a = np.array( [ [1,2,3], [4,5,6] ] )
```

```
>>> a[0]
```

```
array([1, 2, 3])
```

```
>>> a[1]
```

```
array([4, 5, 6])
```

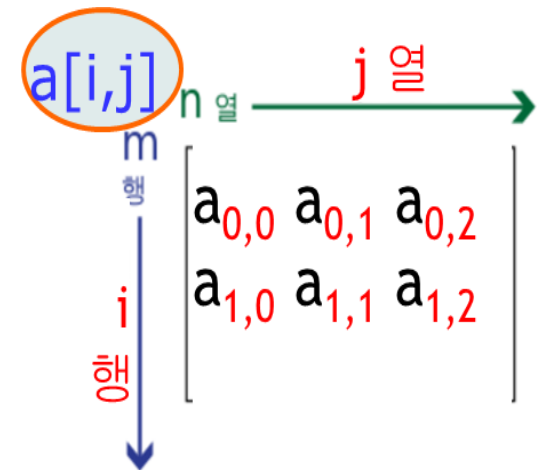
```
>>> a[0,0]
```

```
1
```

```
>>> a[1,2]
```

```
6
```

```
>>> a  
array( [ [1, 2, 3],  
         [4, 5, 6] ] )
```



numpy 2차원 배열의 슬라이싱

- numpy 2차원 배열에서 부분 집합 구하기.
 - 슬라이싱
 - 배열의 각 차원별로 슬라이싱 범위를 지정.
(예) 2차원 배열. 변수[행 슬라이싱, 열 슬라이싱]
 - 정수 인덱싱 (integer indexing)
 - 각 차원별로 선택되어지는 배열 요소의 인덱스들을 일렬로 나열
 - (예) 2차원 배열 **a** 에서
배열 요소 **a[row1, col1], a[row2, col2], ...** 을 추출하려면,

a[[row1, row2, ...], [col1, col2, ...]]

 와 같이 지정.
 - 부울린 인덱싱 (boolean indexing)
 - 부울린 인덱싱 (boolean indexing)을 사용하여
특정 조건의 데이터만 필터링 할 수도 있다.
 - 인덱스로 사용되는 부울형 배열을 만들고,
배열 각 요소의 선택 여부를 **True, False**로 표현하는 방식.
 - **True**인 요소만 추출 함.

numpy 2차원 배열의 슬라이싱

- (예) 2차원 배열 슬라이싱. 변수[행 슬라이싱, 열 슬라이싱]

```
>>> data = [ [ 1, 2, 3, 4, 5],  
              [ 6, 7, 8, 9, 10],  
              [11, 12, 13, 14, 15] ]  
  
>>> a = np.array(data)
```

#행과 열의 슬라이싱 지정

```
>>> a[ 0:2, 0:3 ]  
array([[1, 2, 3],  
       [6, 7, 8]])  
  
>>>
```

행슬라이싱 0:2



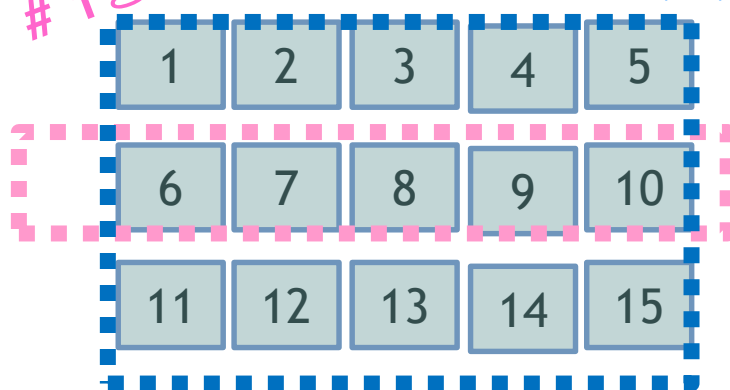
#1행 선택하기

#1행 선택, 모든 열 선택. `a[1]`과 동일

```
>>> a[ 1, : ]  
array([ 6,  7,  8,  9, 10])  
  
>>>
```

1행 선택

열슬라이싱 :



numpy 2차원 배열의 슬라이싱

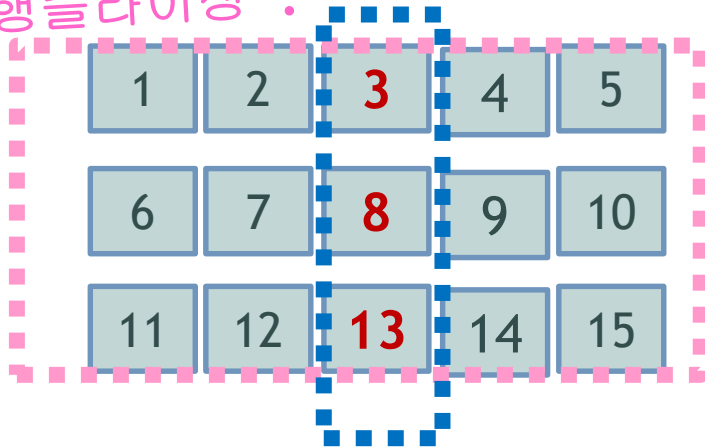
- (예) 2차원 배열 슬라이싱. 변수[행 슬라이싱, 열 슬라이싱]

#2열 선택하기

#모든 행 선택, 2열 선택

```
>>> a[:, 2]  
array([3, 8, 13])  
>>>
```

행슬라이싱 :



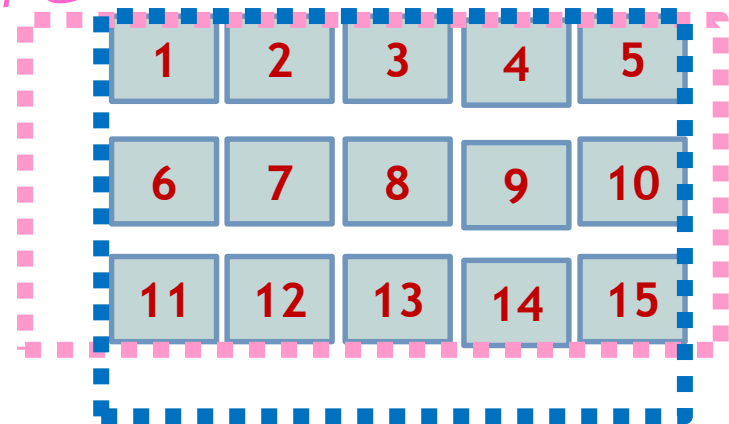
열선택 2

#모든 원소 선택하기

#모든 행 선택, 모든 열 선택

```
>>> a[:, :]  
array([[ 1,  2,  3,  4,  5],  
       [ 6,  7,  8,  9, 10],  
       [11, 12, 13, 14, 15]])  
>>>
```

행슬라이싱 :



열슬라이싱 :

numpy 2차원 배열의 슬라이싱

- numpy 정수 인덱싱 (integer indexing)

- 각 차원별로 선택되어지는 배열 요소의 인덱스들을 일렬로 나열
- 2차원 배열 a에서 `a[ [row1, row2, ...], [col1, col2, ...] ]`
 - 배열 요소 `a[row1, col1]`, `a[row2, col2]`, ... 을 추출
- 예) 배열 요소 `a[0,1]`와 `a[2,4]` 추출하려면 `→ a[ [0,2], [1,4] ]`

행의 열의 인덱스
인덱스

```
>>> data = [  
    [ 1, 2, 3, 4, 5],  
    [ 5, 6, 7, 8, 10],  
    [11, 12, 13, 14, 15] ]
```

```
>>> a = np.array(data)  
>>> a[ [0, 2] , [1, 4] ] # a[0,1]와 a[2,4] 추출  
array([ 2, 15])  
>>> print(a[ [0, 2] , [1, 4] ])  
[ 2 15]  
>>>
```

numpy 2차원 배열의 슬라이싱

- numpy 부울형 인덱싱 (boolean indexing)
 - **부울형 인덱싱 (boolean indexing)**으로 특정 조건의 데이터만 필터링 할 수 있다.
 - 부울형 배열은
 - True, False로 이루어진 배열.
 - 추출하는 배열과 크기가 같아야 함.
 - 원소의 추출 여부를 True, False로 표현.
 - 선택할 원소는 True, 그렇지 않으면 False로 표현.
 - 인덱스로 지정된 부울형 배열의 True와 대응되는 요소만 추출.
 - 부울형 배열은 배열의 관계연산으로 보통 구성한다.
배열의 관계연산은 True, False로 이루어진 배열을 반환, 이것을 인덱스로 사용하여 특정 값을 추출한다.

```
a= np.array( [ [ 1, 2, 3],  
               [ 4, 5, 6],  
               [ 7, 8, 9] ] )
```

```
>>> a%2==0   
array([ [False, True, False],  
       [ True, False, True],  
       [False, True, False] ] )  
>>>
```

True, False로
이루어진
배열

numpy 2차원 배열의 슬라이싱

- 예) numpy배열에서 짝수인 원소만 추출하기.

```
>>> a= np.array( [ [ 1, 2, 3],  
                  [ 4, 5, 6],  
                  [ 7, 8, 9] ] )
```

```
>>> a[ a%2 == 0 ] # boolean indexing  
[ 2 4 6 8 ]      # boolean 배열을 인덱스로 하여,  
                 # 원소가 True인 값만 추출.
```

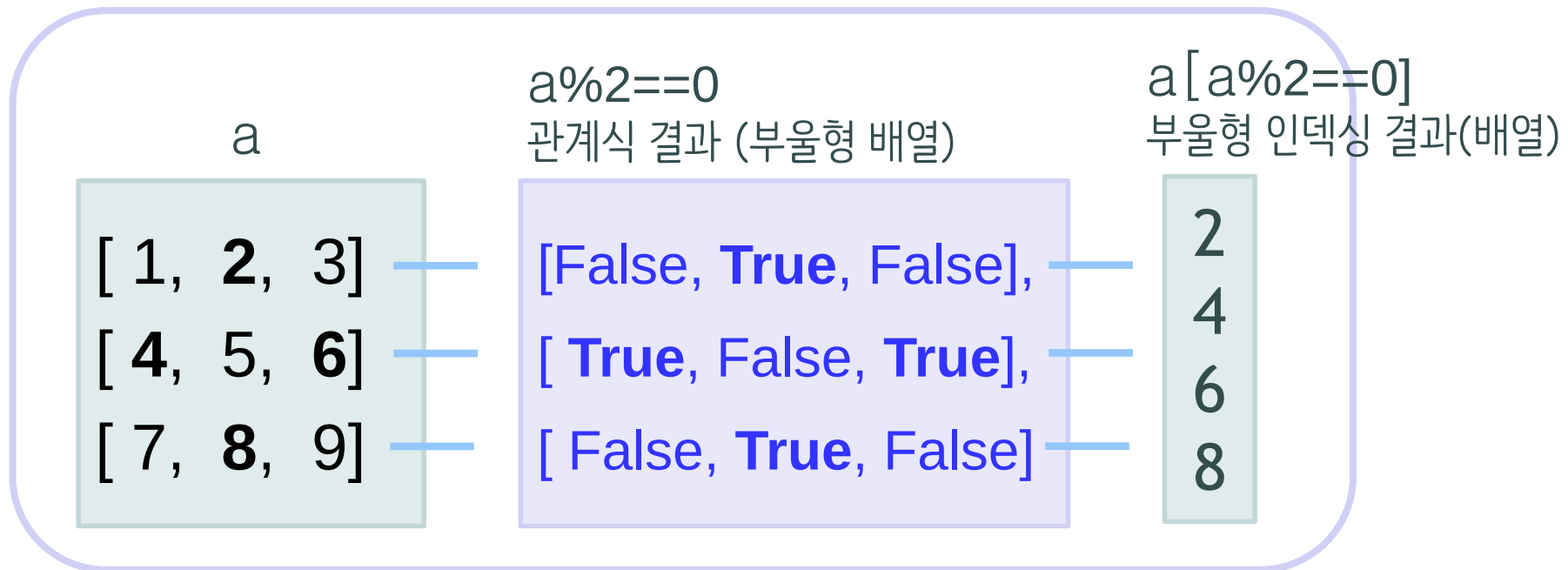
```
>>> a % 2 == 0  
array([ [False,  True, False],  
       [  True, False,  True],  
       [False,  True, False] ] )
```

```
>>>
```

} 부울린 배열 생성.
배열 a의 원소가 짝수이면 True,
홀수이면 False인 배열.

numpy 2차원 배열의 슬라이싱

- `a[a%2==0]`
 - numpy 부울형 인덱싱 (boolean indexing)으로 조건에 맞는 배열의 원소들을 추출한다.
 - 이렇게 구성한 새 배열이 결과 값이다.



True 항목과 매치되는
원소만 추출